

Pascal Grammar

Jawad Ahmed

20th January 2024

This document details a hierarchical EBNF grammar for Pascal per Level 1 of the ISO 7185:1990 standard.

0 Programs

$\langle \text{program} \rangle$	$::= \langle \text{program heading} \rangle \text{' ;' } \langle \text{block} \rangle \text{' . '}$
$\langle \text{program heading} \rangle$	$::= \text{' program' } \langle \text{identifier} \rangle [\langle \text{program param list} \rangle]$
$\langle \text{program param list} \rangle$	$::= \text{' (' } \langle \text{identifier list} \rangle \text{') '}$
$\langle \text{block} \rangle$	$::= [\langle \text{label decl part} \rangle] [\langle \text{const def part} \rangle] [\langle \text{type def part} \rangle] [\langle \text{var decl part} \rangle] \langle \text{routine part} \rangle \langle \text{stmt part} \rangle$
$\langle \text{label decl part} \rangle$	$::= \text{' label' } \langle \text{digit seq} \rangle \{ \text{' , ' } \langle \text{digit seq} \rangle \} \text{' ; '}$
$\langle \text{const def part} \rangle$	$::= \text{' const' } \langle \text{const def} \rangle \text{' ; ' } \{ \langle \text{const def} \rangle \text{' ; ' } \}$
$\langle \text{type def part} \rangle$	$::= \text{' type' } \langle \text{type def} \rangle \text{' ; ' } \{ \langle \text{type def} \rangle \text{' ; ' } \}$
$\langle \text{var decl part} \rangle$	$::= \text{' var' } \langle \text{var decl} \rangle \text{' ; ' } \{ \langle \text{var decl} \rangle \text{' ; ' } \}$
$\langle \text{routine decl part} \rangle$	$::= \{ (\langle \text{proc decl} \rangle \mid \langle \text{func decl} \rangle) \text{' ; ' } \}$
$\langle \text{stmt part} \rangle$	$::= \langle \text{compound stmt} \rangle$

1 Declarations

$\langle \text{const def} \rangle$	$::= \langle \text{ident} \rangle \text{' = ' } \langle \text{const} \rangle$
$\langle \text{type def} \rangle$	$::= \langle \text{ident} \rangle \text{' = ' } \langle \text{type} \rangle$
$\langle \text{var decl} \rangle$	$::= \langle \text{ident list} \rangle \text{' : ' } \langle \text{type} \rangle$
$\langle \text{proc decl} \rangle$	$::= \langle \text{proc heading} \rangle \text{' ; ' } \langle \text{block} \rangle$ $\mid \langle \text{proc heading} \rangle \text{' ; ' } \langle \text{directive} \rangle$ $\mid \langle \text{proc identification} \rangle \text{' ; ' } \langle \text{block} \rangle$
$\langle \text{func decl} \rangle$	$::= \langle \text{func heading} \rangle \text{' ; ' } \langle \text{block} \rangle$ $\mid \langle \text{func heading} \rangle \text{' ; ' } \langle \text{directive} \rangle$ $\mid \langle \text{func identification} \rangle \text{' ; ' } \langle \text{block} \rangle$
$\langle \text{proc heading} \rangle$	$::= \text{' procedure' } \langle \text{ident} \rangle [\langle \text{formal param list} \rangle]$
$\langle \text{proc identification} \rangle$	$::= \text{' procedure' } \langle \text{proc ident} \rangle$
$\langle \text{func heading} \rangle$	$::= \text{' function' } \langle \text{ident} \rangle [\langle \text{formal param list} \rangle] \text{' : ' } \langle \text{result type} \rangle$

$\langle \text{func identification} \rangle ::= \text{'function'} \langle \text{func ident} \rangle$
 $\langle \text{formal param list} \rangle ::= \text{'('} \langle \text{formal param section} \rangle \{ \text{';' } \langle \text{formal param section} \rangle \} \text{'}'$
 $\langle \text{formal param section} \rangle ::= \langle \text{value param spec} \rangle$
 $\quad \quad \quad | \langle \text{var param spec} \rangle$
 $\quad \quad \quad | \langle \text{proc param spec} \rangle$
 $\quad \quad \quad | \langle \text{func param spec} \rangle$

2 Parameters

$\langle \text{value param spec} \rangle ::= \langle \text{ident list} \rangle \text{' : '}$
 $\quad \quad \quad (\langle \text{type ident} \rangle | \langle \text{conformant array schema} \rangle)$
 $\langle \text{var param spec} \rangle ::= \text{'var'} \langle \text{ident list} \rangle \text{' : '}$
 $\quad \quad \quad (\langle \text{type ident} \rangle | \langle \text{conformant array schema} \rangle)$
 $\langle \text{proc param spec} \rangle ::= \langle \text{proc heading} \rangle$
 $\langle \text{func param spec} \rangle ::= \langle \text{func heading} \rangle$
 $\langle \text{conformant array schema} \rangle ::= \langle \text{packed conformant array schema} \rangle$
 $\quad \quad \quad | \langle \text{unpacked conformant array schema} \rangle$
 $\langle \text{packed conformant array schema} \rangle ::= \text{'packed'} \text{'array'} \text{'['} \langle \text{index type spec} \rangle \text{'] 'of' } \langle \text{type ident} \rangle$
 $\langle \text{unpacked conformant array schema} \rangle ::= \text{'array'} \text{'['} \langle \text{index type spec} \rangle \{ \text{';' } \langle \text{index type spec} \rangle \} \text{'] 'of' } (\langle \text{type ident} \rangle | \langle \text{conformant array schema} \rangle)$
 $\langle \text{index type spec} \rangle ::= \langle \text{ident} \rangle \text{' .. ' } \langle \text{ident} \rangle \text{' : ' } \langle \text{ordinal type ident} \rangle$

3 Statements

$\langle \text{compound stmt} \rangle ::= \text{'begin'} \langle \text{stmt seq} \rangle \text{'end'}$
 $\langle \text{stmt seq} \rangle ::= \langle \text{stmt} \rangle \{ \text{';' } \langle \text{stmt} \rangle \}$
 $\langle \text{stmt} \rangle ::= [\langle \text{label} \rangle \text{' : ' }] (\langle \text{simple stmt} \rangle | \langle \text{structured stmt} \rangle)$
 $\langle \text{simple stmt} \rangle ::= \langle \text{empty stmt} \rangle$
 $\quad \quad \quad | \langle \text{assign stmt} \rangle$
 $\quad \quad \quad | \langle \text{proc stmt} \rangle$
 $\quad \quad \quad | \langle \text{goto stmt} \rangle$
 $\langle \text{structured stmt} \rangle ::= \langle \text{compound stmt} \rangle$
 $\quad \quad \quad | \langle \text{conditional stmt} \rangle$
 $\quad \quad \quad | \langle \text{repetitive stmt} \rangle$
 $\quad \quad \quad | \langle \text{with stmt} \rangle$
 $\langle \text{conditional stmt} \rangle ::= \langle \text{if stmt} \rangle$
 $\quad \quad \quad | \langle \text{case stmt} \rangle$
 $\langle \text{repetitive stmt} \rangle ::= \langle \text{while stmt} \rangle$
 $\quad \quad \quad | \langle \text{repeat stmt} \rangle$
 $\quad \quad \quad | \langle \text{for stmt} \rangle$
 $\langle \text{empty stmt} \rangle ::=$

$\langle \text{assign stmt} \rangle$	$::= (\langle \text{var} \rangle \mid \langle \text{func ident} \rangle) \text{ ':=' } \langle \text{expr} \rangle$
$\langle \text{procedure stmt} \rangle$	$::= \langle \text{proc ident} \rangle [\langle \text{actual param list} \rangle \mid \langle \text{write param list} \rangle]$
$\langle \text{goto stmt} \rangle$	$::= \text{'goto'} \langle \text{label} \rangle$
$\langle \text{if stmt} \rangle$	$::= \text{'if'} \langle \text{bool expr} \rangle \text{'then'} \langle \text{stmt} \rangle [\text{'else'} \langle \text{stmt} \rangle]$
$\langle \text{case stmt} \rangle$	$::= \text{'case'} \langle \text{case index} \rangle \text{'of'} \langle \text{case} \rangle \{ \text{';' } \langle \text{case} \rangle \} [\text{';' }] \text{'end'}$
$\langle \text{repeat stmt} \rangle$	$::= \text{'repeat'} \langle \text{stmt seq} \rangle \text{'until'} \langle \text{bool expr} \rangle$
$\langle \text{while stmt} \rangle$	$::= \text{'while'} \langle \text{bool expr} \rangle \text{'do'} \langle \text{stmt} \rangle$
$\langle \text{for stmt} \rangle$	$::= \text{'for'} \langle \text{control var} \rangle \text{' := ' } \langle \text{ordinal expr} \rangle (\text{'to'} \mid \text{'downto'})$ $\langle \text{ordinal expr} \rangle \text{'do'} \langle \text{stmt} \rangle$
$\langle \text{with stmt} \rangle$	$::= \text{'with'} \langle \text{record var list} \rangle \text{'do'} \langle \text{stmt} \rangle$
$\langle \text{record var list} \rangle$	$::= \langle \text{record var} \rangle \{ \text{' , ' } \langle \text{record var} \rangle \}$
$\langle \text{case index} \rangle$	$::= \langle \text{ordinal expr} \rangle$
$\langle \text{case} \rangle$	$::= \langle \text{const} \rangle \{ \text{' , ' } \langle \text{const} \rangle \} \text{' : ' } \langle \text{stmt} \rangle$
$\langle \text{control var} \rangle$	$::= \langle \text{var ident} \rangle$

4 Types

$\langle \text{type} \rangle$	$::= \langle \text{simple type} \rangle$ $\mid \langle \text{structured type} \rangle$ $\mid \langle \text{pointer type} \rangle$
$\langle \text{simple type} \rangle$	$::= \langle \text{ordinal type} \rangle$ $\mid \langle \text{real type ident} \rangle$
$\langle \text{structured type} \rangle$	$::= [\text{'packed'}] \langle \text{unpacked structured type} \rangle$ $\mid \langle \text{structured type ident} \rangle$
$\langle \text{pointer type} \rangle$	$::= (\text{'^'} \mid \text{'\uparrow'}) \langle \text{domain type} \rangle$ $\mid \langle \text{pointer type ident} \rangle$
$\langle \text{ordinal type} \rangle$	$::= \langle \text{enumerated type} \rangle$ $\mid \langle \text{subrange type} \rangle$ $\mid \langle \text{ordinal type ident} \rangle$
$\langle \text{unpacked structured type} \rangle$	$::= \langle \text{array type} \rangle$ $\mid \langle \text{record type} \rangle$ $\mid \langle \text{set type} \rangle$ $\mid \langle \text{file type} \rangle$
$\langle \text{domain type} \rangle$	$::= \langle \text{type ident} \rangle$
$\langle \text{enumerated type} \rangle$	$::= \text{'(' } \langle \text{ident list} \rangle \text{')'}$
$\langle \text{subrange type} \rangle$	$::= \langle \text{const} \rangle \text{'..'} \langle \text{const} \rangle$
$\langle \text{array type} \rangle$	$::= \text{'array'} [\text{'[' } \langle \text{index type} \rangle \{ \text{' , ' } \langle \text{index type} \rangle \} \text{']' } \text{'of'} \langle \text{component type} \rangle$

⟨record type⟩	::= 'record' ⟨field list⟩ 'end'
⟨set type⟩	::= 'set' 'of' ⟨base type⟩
⟨file type⟩	::= 'file' 'of' ⟨component type⟩
⟨index type⟩	::= ⟨ordinal type⟩
⟨component type⟩	::= ⟨type⟩
⟨base type⟩	::= ⟨ordinal type⟩
⟨result type⟩	::= ⟨ordinal type ident⟩ ⟨real type ident⟩ ⟨pointer type ident⟩
⟨field list⟩	::= [(⟨fixed part⟩ [';' ⟨variant part⟩] ⟨variant part⟩) [';']]
⟨fixed part⟩	::= ⟨record section⟩ { ';' ⟨record section⟩ }
⟨variant part⟩	::= 'case' ⟨variant selector⟩ 'of' ⟨variant⟩ { ';' ⟨variant⟩ }
⟨record section⟩	::= ⟨ident list⟩ ':' ⟨type⟩
⟨variant selector⟩	::= [⟨tag field⟩ ':'] ⟨tag type⟩
⟨variant⟩	::= ⟨const⟩ { ',' ⟨const⟩ } ':' '(' ⟨field list⟩ ')'
⟨tag type⟩	::= ⟨ordinal type ident⟩
⟨tag field⟩	::= ⟨ident⟩
⟨const⟩	::= [⟨sign⟩] (⟨unsigned const⟩ ⟨const ident⟩)

5 Expressions

⟨expr⟩	::= ⟨simple expr⟩ [⟨relational op⟩ ⟨simple expression⟩]
⟨simple expr⟩	::= [⟨sign⟩] ⟨term⟩ { ⟨adding op⟩ ⟨term⟩ }
⟨term⟩	::= ⟨factor⟩ { ⟨multiplying op⟩ ⟨factor⟩ }
⟨factor⟩	::= ⟨unsigned const⟩ ⟨bound ident⟩ ⟨var⟩ ⟨set constructor⟩ ⟨function designator⟩ 'not' ⟨factor⟩ '(' ⟨expr⟩ ')'
⟨relational op⟩	::= '=' '⟨' '<' '>' '≤' '≥' 'in'
⟨adding op⟩	::= '+' '-' 'or'
⟨multiplying op⟩	::= '*' '/' 'div' 'mod' 'and'
⟨unsigned const⟩	::= ⟨unsigned number⟩ ⟨char string⟩ ⟨const ident⟩ 'nil'

$\langle \text{function designator} \rangle ::= \langle \text{function ident} \rangle [\langle \text{actual param list} \rangle]$
 $\langle \text{var} \rangle ::= \langle \text{entire var} \rangle$
 $\quad \quad \quad | \langle \text{component var} \rangle$
 $\quad \quad \quad | \langle \text{identified var} \rangle$
 $\quad \quad \quad | \langle \text{buffer var} \rangle$
 $\langle \text{entire var} \rangle ::= \langle \text{var ident} \rangle$
 $\langle \text{component var} \rangle ::= \langle \text{indexed var} \rangle$
 $\quad \quad \quad | \langle \text{field designator} \rangle$
 $\langle \text{identified var} \rangle ::= \langle \text{pointer var} \rangle (\text{'^'} | \text{'\uparrow'})$
 $\langle \text{buffer var} \rangle ::= \langle \text{file var} \rangle (\text{'^'} | \text{'\uparrow'})$
 $\langle \text{indexed var} \rangle ::= \langle \text{array var} \rangle [\langle \text{index} \rangle \{ \text{' , ' } \langle \text{index} \rangle \}]$
 $\langle \text{field designator} \rangle ::= [\langle \text{record var} \rangle \text{' . ' }] \langle \text{field ident} \rangle$
 $\langle \text{set constructor} \rangle ::= [\text{' [' } [\langle \text{element description} \rangle \{ \text{' , ' } \langle \text{element description} \rangle \}] \text{'] '}$
 $\langle \text{element description} \rangle ::= \langle \text{ordinal expr} \rangle [\text{' .. ' } \langle \text{ordinal expr} \rangle]$
 $\langle \text{actual param list} \rangle ::= \text{' (' } \langle \text{actual param} \rangle \{ \text{' , ' } \langle \text{actual param} \rangle \} \text{') '}$
 $\langle \text{actual param} \rangle ::= \langle \text{expr} \rangle$
 $\quad \quad \quad | \langle \text{var} \rangle$
 $\quad \quad \quad | \langle \text{proc ident} \rangle$
 $\quad \quad \quad | \langle \text{func ident} \rangle$
 $\langle \text{write param list} \rangle ::= \text{' (' } (\langle \text{file var} \rangle | \langle \text{write param} \rangle) \{ \text{' , ' } \langle \text{write param} \rangle \} \text{') '}$
 $\langle \text{write param} \rangle ::= \langle \text{expr} \rangle [\text{' : ' } \langle \text{integer expr} \rangle [\text{' : ' } \langle \text{integer expr} \rangle]]$

6 Tokens

$\langle \text{array var} \rangle ::= \langle \text{var} \rangle$
 $\langle \text{record var} \rangle ::= \langle \text{var} \rangle$
 $\langle \text{file var} \rangle ::= \langle \text{var} \rangle$
 $\langle \text{pointer var} \rangle ::= \langle \text{var} \rangle$
 $\langle \text{integer expr} \rangle ::= \langle \text{ordinal expr} \rangle$
 $\langle \text{bool expr} \rangle ::= \langle \text{ordinal expr} \rangle$
 $\langle \text{index} \rangle ::= \langle \text{ordinal expr} \rangle$
 $\langle \text{ordinal expr} \rangle ::= \langle \text{expr} \rangle$
 $\langle \text{pointer type ident} \rangle ::= \langle \text{type ident} \rangle$
 $\langle \text{structured type ident} \rangle ::= \langle \text{type ident} \rangle$
 $\langle \text{ordinal type ident} \rangle ::= \langle \text{type ident} \rangle$

⟨real type ident⟩	::= ⟨type ident⟩
⟨const ident⟩	::= ⟨ident⟩
⟨type ident⟩	::= ⟨ident⟩
⟨var ident⟩	::= ⟨ident⟩
⟨field ident⟩	::= ⟨ident⟩
⟨proc ident⟩	::= ⟨ident⟩
⟨func ident⟩	::= ⟨ident⟩
⟨bound ident⟩	::= ⟨ident⟩
⟨unsigned number⟩	::= ⟨unsigned integer⟩ ⟨unsigned real⟩
⟨ident list⟩	::= ⟨ident⟩ { ‘,’ ⟨ident⟩ }
⟨ident⟩	::= ⟨letter⟩ { ⟨letter⟩ ⟨digit⟩ }
⟨directive⟩	::= ⟨letter⟩ { ⟨letter⟩ ⟨digit⟩ }
⟨label⟩	::= ⟨digit seq⟩
⟨unsigned integer⟩	::= ⟨digit seq⟩
⟨unsigned real⟩	::= ⟨digit seq⟩ ‘.’ ⟨digit seq⟩ [‘e’ ⟨scale factor⟩] ⟨digit seq⟩ ‘e’ ⟨scale factor⟩
⟨scale factor⟩	::= [⟨sign⟩] ⟨digit seq⟩
⟨sign⟩	::= ‘+’ ‘-’
⟨char string⟩	::= ‘’ ⟨string element⟩ { ⟨string element⟩ } ‘’
⟨digit seq⟩	::= ⟨digit⟩ { ⟨digit⟩ }
⟨letter⟩	::= ‘a’ ‘b’ ‘c’ ‘d’ ‘e’ ‘f’ ‘g’ ‘h’ ‘i’ ‘j’ ‘k’ ‘l’ ‘m’ ‘n’ ‘o’ ‘p’ ‘q’ ‘r’ ‘s’ ‘t’ ‘u’ ‘v’ ‘w’ ‘x’ ‘y’ ‘z’
⟨digit⟩	::= ‘0’ ‘1’ ‘2’ ‘3’ ‘4’ ‘5’ ‘6’ ‘7’ ‘8’ ‘9’
⟨string element⟩	::= ‘ ’ ⟨any char except ‘ ’⟩

7 Key

Abbreviation	Expansion
decl	declaration
def	definition
const	constant
var	variable
routine	procedure and function
stmt	statement
seq	sequence
ident	identifier
proc	procedure
func	function
param	parameter
spec	specification
assign	assignment
bool	Boolean
op	operator
char	character